

Sheharzad Salahuddin

Full-Stack Engineer · React / TypeScript · Postgres · AI-Assisted Dev

PORTFOLIO BRIEF

How I Use AI Coding Tools in My Daily Workflow

I use Claude Code, Cursor, and Codex as a fast pair — never as autopilot.

My loop on every ticket:

1. Inspect first.

Before writing anything, I have the AI map where the relevant logic actually lives and what depends on it. This prevents "fix one thing, break another" in an unfamiliar codebase.

2. Plan small.

I ask for a scoped plan tied to the ticket's acceptance criteria — not a rewrite. Smallest correct change wins.

3. Implement + read every line.

The AI drafts; I review all of it. No blind paste. I reject anything I can't explain.

4. Verify.

I run the app, test against the acceptance criteria, and check the edge cases the ticket implies.

5. Refine + hand off.

Small, reviewable diff plus a short written summary so QA and the next dev aren't guessing.

Why it matters for maintenance work: on live products, velocity is worthless if it creates regressions. This loop gives speed *and* safety — small diffs, clear handoffs, fewer surprises for QA.

Debugging Walkthrough: Tracing a Bug From UI to Database

Symptom: A data table rendered stale/incorrect values after an update, even though the save "succeeded."

How I traced it (UI → API → data layer):

- Confirmed the UI was sending the correct payload (network tab / request inspection).
- Checked the API response — the write succeeded, but the returned object didn't reflect the change.
- Traced to the data layer: the read was hitting a cached/earlier value, so the refetch returned old data.

Fix: Invalidated the cached query on mutation success so the table refetched fresh data. Small, contained change — no schema or API contract changes needed.

Verification: Reproduced the original bug, applied the fix, confirmed the table updated immediately, and checked that the adjacent flows sharing the same query weren't affected.

Takeaway: Most "frontend" bugs in data-heavy SaaS are actually data-flow bugs. I trace the full path before touching code.

Selected Relevant Work

Live, working projects that map directly to this role. All links are public.

Eden Avenue Management — Admin dashboard & workflow SaaS

Next.js (App Router) + TypeScript + Prisma + **PostgreSQL** + NextAuth. Role-based access and a maintenance workflow (create → assign → status → comments) with unit management, notices, and real-time stats. Shipped as an installable PWA.

(Maps to: admin dashboards, crew/contractor/PO workflows, Postgres.)

Live: <https://edenavenue.vercel.app/>

Code: <https://github.com/sheharzad-developer/edenavenue>

LSLS School Management System — Admin dashboard + REST API

Next.js + TypeScript + Prisma + **PostgreSQL** + NextAuth + REST API.

(Maps to: admin dashboards, backend/API + Postgres, data wiring.)

Live: <https://lsismanagementsystemschoo.netlify.app/>

AuraAi — AI Wellness Companion — AI feature + Supabase

Next.js / React 19 + Google Gemini + **Server-Sent Events streaming** +

Supabase + Stripe.

(Maps to: AI features, chatbot flows, Supabase.)

Live: <https://ai-mental-health-chat-app.vercel.app/>

ChatSmart AI — RAG / vector retrieval

Python + LangChain + Google Gemini + **FAISS vector store** + HuggingFace.

(Maps to: RAG, pgvector-style retrieval, content ingestion.)

Code: <https://github.com/sheharzad-developer/chatsmart-ai>

AI Chatbot Platform — Chatbot backend

FastAPI + Python + LangChain + MongoDB + MCP + Langfuse.

(Maps to: chatbot flows, backend AI features.)

Code: https://github.com/sheharzad-developer/Chatbot_Internship

Daggys Cafe — Full-stack + Supabase + realtime

Next.js + TypeScript + **Supabase** + Stripe + Socket.io.

(Maps to: Supabase, forms/tables, realtime data flows.)

Live: <https://daggys-cafe.vercel.app/>

Code: <https://github.com/sheharzad-developer/daggys-cafe>